Python Dictionary Methods

Complete Guide with Syntax & Examples

What is a Dictionary?

A **Dictionary** is an ordered (Python 3.7+), mutable collection of **key-value pairs**. Keys must be unique and immutable; values can be any type. Duplicates overwrite the earlier value.

```
d = {'name': 'Alice', 'age': 25, 'city': 'Dhaka'}
```

Output:

```
{'name': 'Alice', 'age': 25, 'city': 'Dhaka'}
```

ACCESSING & READING METHODS

1. get()

Returns the value for a key. Returns None (or a default) if key is NOT found — safer than d[key] which raises KeyError.

Syntax: `dict.get(key, default=None)`

```
d = {'name': 'Alice', 'age': 25}
print(d.get('name'))
print(d.get('salary', 0)) # key missing, use default
```

Output:

```
Alice
0
```

2. keys()

Returns a view object of ALL keys in the dictionary. Updates automatically when the dict changes.

Syntax: `dict.keys()`

```
d = {'name': 'Alice', 'age': 25}
print(d.keys())
```

Output:

```
dict_keys(['name', 'age'])
```

Tip: Convert to list with list(d.keys())

3. values()
Returns a view object of ALL values in the dictionary.

Syntax: `dict.values()`

```
d = {'name': 'Alice', 'age': 25}
print(d.values())
```

Output:

```
dict_values(['Alice', 25])
```

4. items()
Returns a view object of ALL key-value pairs as tuples. Very useful for looping.

Syntax: `dict.items()`

```
d = {'name': 'Alice', 'age': 25}
print(d.items())
for k, v in d.items():
print(k, '->', v)
```

Output:

```
dict_items([('name', 'Alice'), ('age', 25)])
name -> Alice
age -> 25
```

ADDING & MODIFYING METHODS

5. update()
Adds or updates multiple key-value pairs from another dictionary or iterable of pairs.

Syntax: `dict.update(other_dict / iterable)`

```
d = {'name': 'Alice'}
d.update({'age': 25, 'city': 'Dhaka'})
print(d)
```

Output:

```
{'name': 'Alice', 'age': 25, 'city': 'Dhaka'}
```

6. setdefault()
Returns the value of a key. If key does NOT exist, inserts it with the given default value.

Syntax: `dict.setdefault(key, default=None)`

```
d = {'name': 'Alice'}
d.setdefault('age', 0) # key missing, inserts it
d.setdefault('name', 'Bob') # key exists, no change
print(d)
```

Output:

```
{'name': 'Alice', 'age': 0}
```

Useful to avoid overwriting existing values.

REMOVING METHODS

7. pop()

Removes a key and RETURNS its value. Raises KeyError if key not found (unless default given).

Syntax: `dict.pop(key, default)`

```
d = {'name': 'Alice', 'age': 25}

val = d.pop('age')

print(val)

print(d)

Output:

25

{'name': 'Alice'}
```

8. popitem()

Removes and returns the LAST inserted key-value pair as a tuple. Raises KeyError if dict is empty.

Syntax: `dict.popitem()`

```
d = {'name': 'Alice', 'age': 25, 'city': 'Dhaka'}

item = d.popitem()

print(item)

print(d)

Output:

('city', 'Dhaka')

{'name': 'Alice', 'age': 25}
```

In Python 3.7+, always removes the last inserted item (LIFO order).

9. clear()

Removes ALL key-value pairs from the dictionary. The dictionary becomes empty {}.

Syntax: `dict.clear()`

```
d = {'name': 'Alice', 'age': 25}

d.clear()

print(d)

Output:

{}
```

COPYING METHOD

10. copy()

Returns a SHALLOW copy of the dictionary. Changes to the copy do not affect the original (for top-level keys).

Syntax: `dict.copy()`

```
d = {'name': 'Alice', 'age': 25}
d2 = d.copy()
d2['age'] = 99
print(d) # original unchanged
print(d2)
```

Output:

```
{'name': 'Alice', 'age': 25}
{'name': 'Alice', 'age': 99}
```

CREATING DICTIONARIES

11. fromkeys()

CLASS METHOD — Creates a new dictionary from a sequence of keys, all set to the same value.

Syntax: `dict.fromkeys(keys, value=None)`

```
keys = ['a', 'b', 'c']
d = dict.fromkeys(keys, 0)
print(d)
```

Output:

```
{'a': 0, 'b': 0, 'c': 0}
```

Useful for initialising a dict with default values.

USEFUL OPERATIONS (Not Methods but Important!)

len(d)

```
d = {'a':1, 'b':2, 'c':3}
print(len(d))
```

Output:

```
3
```

in (key check)

```
d = {'name': 'Alice'}
print('name' in d)
print('age' in d)
```

Output:

```
True
False
```

del d[key]

```
d = {'name': 'Alice', 'age': 25}
del d['age']
print(d)
```

Output:

```
{'name': 'Alice'}
```

dict comprehension

```
squares = {x: x**2 for x in range(1,6)}
print(squares)
```

Output:

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

Quick Reference Summary

Method	What it does	Returns / Effect
`get(key, default)`	Return value safely	Value or default
`keys()`	All keys view	dict_keys object
`values()`	All values view	dict_values object
`items()`	All key-value pairs	dict_items of tuples
`update(other)`	Add/overwrite pairs	Modifies in-place
`setdefault(key, val)`	Insert if key missing	Value (existing or new)
`pop(key)`	Remove & return value	Value / KeyError
`popitem()`	Remove last pair	Tuple (key, value)
`clear()`	Remove all pairs	Empty dict {}
`copy()`	Shallow copy	New dictionary
`fromkeys(keys, val)`	Create from key list	New dictionary

KEY FACTS: Dictionary = Mutable + Key-Value pairs + Keys must be unique | Use get() to avoid KeyError | items() is best for looping | fromkeys() is a class method called on dict, not an instance